

CS 521 ML & Compilers – MP 1

Part 1: CPU

2. CPU ablation study

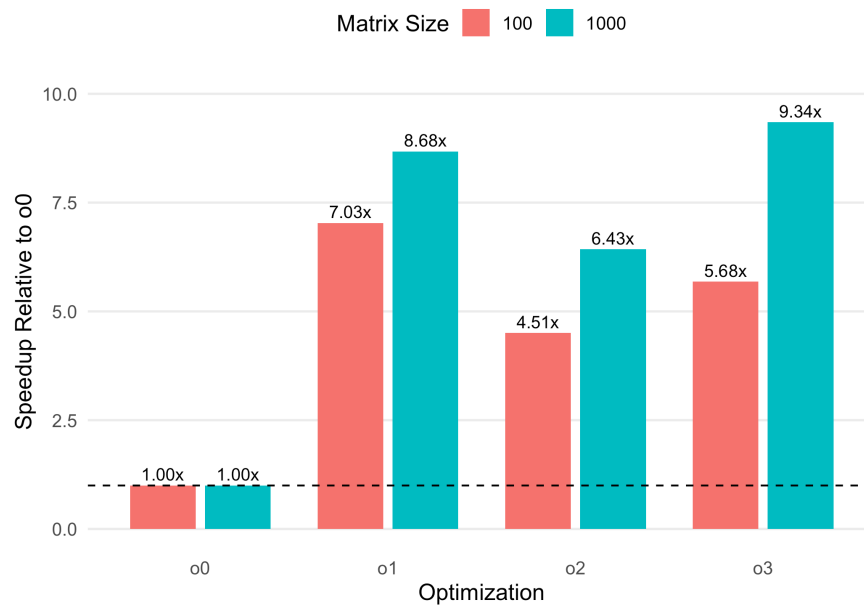


Figure 1: speedup comparison of optimizations relative to naive implementation on CPU

Loop reordering to i-k-j provided a large improvement in performance due to spatial locality since contiguous elements of both B and C are accessed. Tiling does not appear to improve this on the hardware this was benchmarked on (Colab Intel Xeon CPU instance), but did produce more pronounced benefits during development on an Apple M4 chip. The parallelization and SIMD implemented in o3 appear to be more beneficial at larger matrix sizes ($N = M = K = 1000$) since the overhead cost of multithreading can be amortized. The benefit was similarly more pronounced on development hardware since more cores were available for parallelization.

3. CPU scalability study

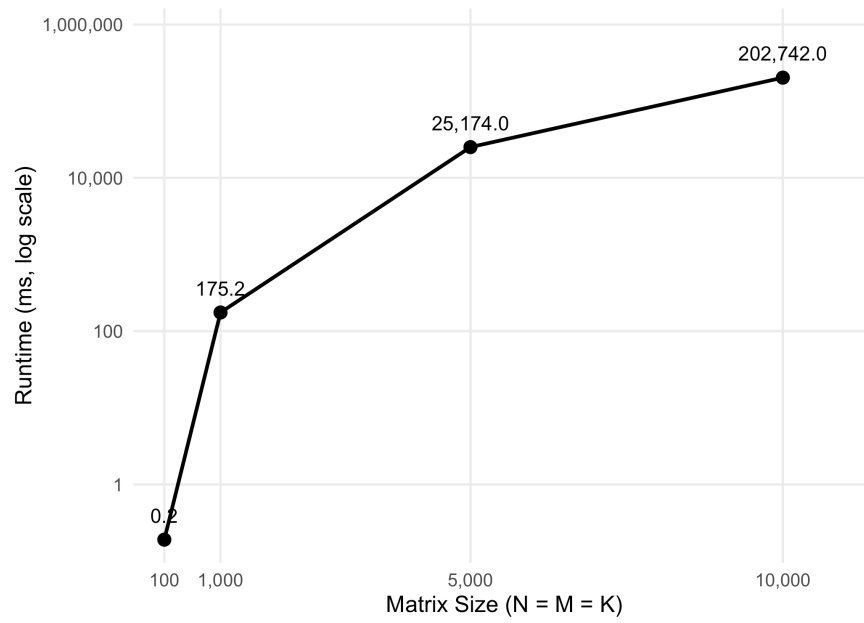


Figure 2: runtime of o4 optimization for different matrix sizes on CPU

Measured runtimes are consistent with the expected GEMM time complexity of $O(N^3)$ and grow rapidly as matrix size increases. In particular, consider the increase in runtime from $N = 5000$ to $N = 10000$: $\frac{202742}{25174} \approx 8.053$ which is very close to the expected increase of $\left(\frac{10000}{5000}\right)^3 = 8$.

2. GPU ablation study

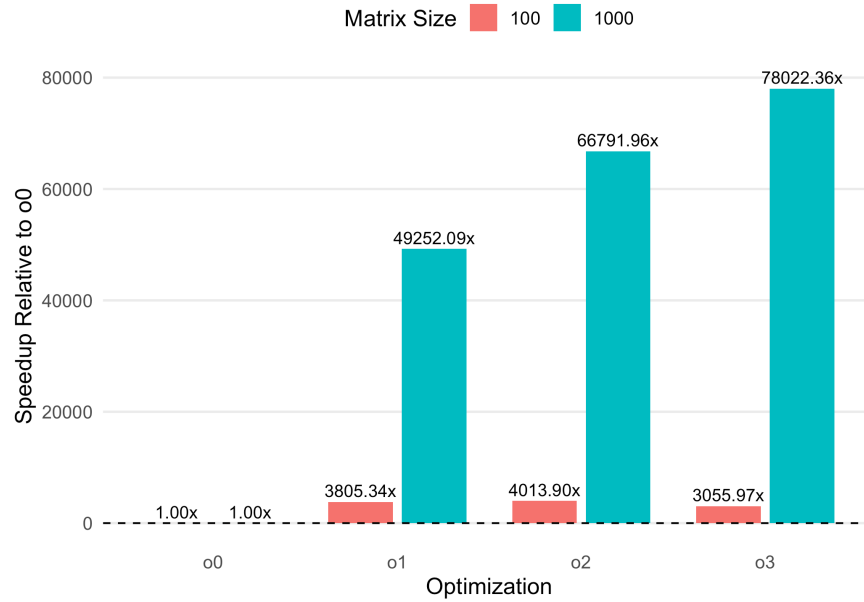


Figure 3: speedup comparison of optimizations relative to naive implementation on T4 GPU

The naive o0 kernel is very slow since it makes a single GPU thread serially perform the entire GEMM computation without leveraging any of the hardware's parallel capabilities. o1 achieves a significant speed up on top of that by mapping each output element to a separate thread. The usage of fast shared memory tiling in o2 earns further speedups on top of o1.

The tile size was tuned for o3. Out of sizes 8, 16, 24, 28, and 32, the tile size of 32 was empirically found to be the most optimal. In addition, o3 also makes uses of `#pragma unroll` to unroll the loop that always run 32 times.

While o3 was slightly outperformed by o1 and o2 at small matrix sizes, it appears to offer significant speedups for larger matrix sizes like $N = 1000$.

3. GPU scalability study

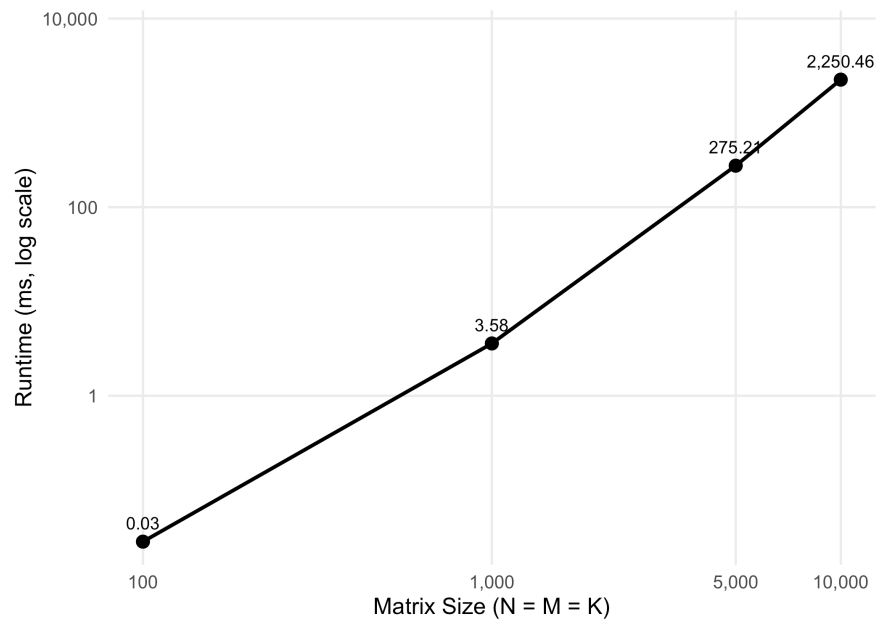


Figure 4: runtime of o3 optimization for different matrix sizes on T4 GPU

Runtime scales approximately cubically, consistent with the expected $O(N^3)$ computational complexity of GEMM. Doubling the matrix dimension from $N = 5000$ to $N = 10000$ increases runtime from 275.21 ms to 2250.46 ms, a factor of approximately 8.17x, close to the expected 8x increase.

4. Comparison to cuBLAS

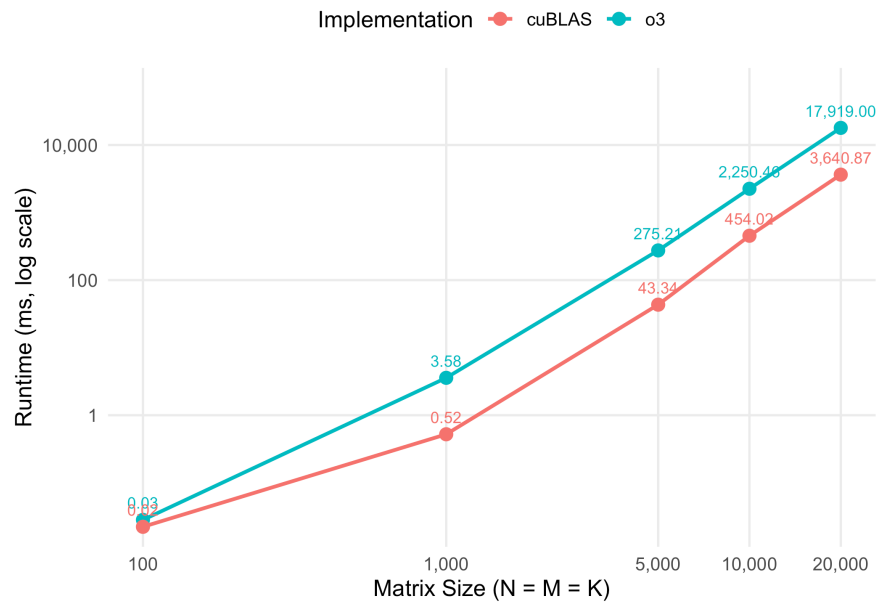


Figure 5: runtime of o3 optimization for different matrix sizes compared to cuBLAS SGEMM

cuBLAS's implementation of the GEMM kernel outperforms o3 at every tested matrix size and is about 5 times faster at $N = 20000$. To narrow this gap, a possible further implementation is register tiling which involves making each thread compute a tile of values rather than a single value in the output matrix. This will minimize the amount of pressure put on shared memory by exploiting spatial locality and reducing synchronizations.

LLM interaction log

Can I get a brief overview of cuda?

CUDA (Compute Unified Device Architecture) is NVIDIA's parallel computing platform and programming model for using GPUs for general-purpose computing, not just graphics.

Core idea: GPUs have thousands of small cores designed for parallel work, while CPUs have fewer, more powerful cores for sequential work. CUDA lets developers write code that offloads parallelizable tasks to the GPU.

Key concepts:

Kernels: Functions written in CUDA C/C++ (an extension of C/C++) that run on the GPU, executed by many threads in parallel

Threads, blocks, grids: Threads are grouped into blocks, and blocks into a grid – this hierarchy maps onto the GPU's hardware structure

Host vs. device: "Host" refers to the CPU and its memory; "device" refers to the GPU and its memory. Data typically needs to be copied from host to device, processed, then copied back

Memory hierarchy: Global, shared, and local memory with different speed/scope tradeoffs – managing this well is key to performance

Why it matters:

It's the dominant platform for GPU-accelerated computing, especially in deep learning (PyTorch, TensorFlow rely heavily on it under the hood), scientific computing, and simulations

Libraries like cuDNN, cuBLAS, and Thrust build on top of CUDA for common tasks

It's proprietary to NVIDIA hardware – AMD's equivalent is ROCm, and there are portability layers like OpenCL or SYCL for cross-vendor code

Basic workflow:

- Allocate memory on the GPU
- Copy input data from CPU to GPU
- Launch a kernel to process data in parallel
- Copy results back from GPU to CPU
- Free GPU memory

Want me to go deeper into any part – like writing a simple kernel example, memory management, or how it fits into ML frameworks?

Yes introduce the technical details, how do I write cuda?

Drafting an introductory guide to CUDA programming.

Drafting an introductory guide to CUDA programming.

Here's how to actually write CUDA code, starting with the toolchain and a working example.

Setup

You need:

- An NVIDIA GPU
- The CUDA Toolkit (includes nvcc, the compiler)
- Files typically use .cu extension

Compile with:
bash

```
nvcc myprogram.cu -o myprogram
```

Anatomy of a CUDA program

1. Kernel functions (__global__)

A kernel is a function that runs on the GPU, launched from the CPU:
cpp

```
__global__ void addVectors(float *a, float *b, float *c, int n) {  
    int i = blockIdx.x * blockDim.x + threadIdx.x;  
    if (i < n) {  
        c[i] = a[i] + b[i];  
    }  
}
```

Key qualifiers:

- __global__ – runs on GPU, callable from CPU (a kernel)
- __device__ – runs on GPU, callable only from GPU code
- __host__ – runs on CPU (the default if unmarked)

2. Thread indexing

This is the part that trips people up most. Each thread needs to figure out "which piece of data am I responsible for?"

- threadIdx.x – thread's index within its block
- blockIdx.x – block's index within the grid
- blockDim.x – number of threads per block

The formula $\text{blockIdx.x} * \text{blockDim.x} + \text{threadIdx.x}$ gives each thread a unique global index. Think of it like flattening a 2D grid coordinate into a 1D array offset.

3. Memory management

GPU and CPU have separate memory spaces. You must explicitly move data:
cpp

```
cudaMalloc(&d_a, size);           // allocate on GPU  
cudaMemcpy(d_a, h_a, size, cudaMemcpyHostToDevice); // copy CPU -> GPU  
// ... run kernel ...  
cudaMemcpy(h_c, d_c, size, cudaMemcpyDeviceToHost); // copy GPU -> CPU  
cudaFree(d_a);                   // free GPU memory
```

(Convention: h_ prefix for host pointers, d_ for device pointers.)

Full working example: vector addition

cpp

```
#include <cuda_runtime.h>  
#include <stdio.h>
```

```
__global__ void addVectors(float *a, float *b, float *c, int n) {  
    int i = blockIdx.x * blockDim.x + threadIdx.x;
```

```

    if (i < n) {
        c[i] = a[i] + b[i];
    }
}

int main() {
    int n = 1 << 20; // ~1 million elements
    size_t size = n * sizeof(float);

    // Allocate host memory
    float *h_a = (float*)malloc(size);
    float *h_b = (float*)malloc(size);
    float *h_c = (float*)malloc(size);

    for (int i = 0; i < n; i++) {
        h_a[i] = 1.0f;
        h_b[i] = 2.0f;
    }

    // Allocate device memory
    float *d_a, *d_b, *d_c;
    cudaMalloc(&d_a, size);
    cudaMalloc(&d_b, size);
    cudaMalloc(&d_c, size);

    // Copy inputs to device
    cudaMemcpy(d_a, h_a, size, cudaMemcpyHostToDevice);
    cudaMemcpy(d_b, h_b, size, cudaMemcpyHostToDevice);

    // Launch kernel: 256 threads per block
    int threadsPerBlock = 256;
    int blocksPerGrid = (n + threadsPerBlock - 1) / threadsPerBlock;
    addVectors<<<blocksPerGrid, threadsPerBlock>>>(d_a, d_b, d_c, n);

    // Copy result back
    cudaMemcpy(h_c, d_c, size, cudaMemcpyDeviceToHost);

    printf("c[0] = %f (expected 3.0)\n", h_c[0]);

    cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);
    free(h_a); free(h_b); free(h_c);
    return 0;
}

```

The <<<blocksPerGrid, threadsPerBlock>>> syntax is the kernel launch – it tells the GPU how many blocks and threads to spin up.
Things you'll run into quickly

Error checking: CUDA calls return error codes silently by default. Wrap calls with a macro that checks `cudaGetLastError()`, or you'll debug blind.

Synchronization: `cudaDeviceSynchronize()` blocks the CPU until the GPU finishes – useful for timing and debugging, since kernel launches are asynchronous.

Shared memory: `__shared__` declares memory shared within a block, much faster than global memory – critical for performance tuning (e.g., matrix multiplication tiling).

Occupancy: choosing threads-per-block (usually 128–1024, multiples of 32 – the "warp size") affects how well you use the GPU's resources.

Want me to walk through a more advanced example next – like shared memory tiling for matrix multiplication, or how this connects to what PyTorch does under the hood?

Explain coalescing in more detail

Memory Coalescing

Coalescing is about how threads in a warp (a group of 32 threads that execute in lockstep) access global memory. Get it right and you get massive bandwidth; get it wrong and the same kernel can run several times slower with zero code logic changes – just different access patterns.

The hardware reality

Global memory (GPU DRAM) isn't accessed one byte or one thread at a time. The memory controller fetches data in fixed-size chunks – typically 32-byte, 64-byte, or 128-byte transactions (aligned segments), depending on the GPU architecture and cache line size.

When a warp executes a memory instruction, all 32 threads issue their memory requests simultaneously. The hardware looks at all 32 addresses at once and tries to service them with as few memory transactions as possible.

Coalesced access: if the 32 threads request addresses that fall within one (or a few) contiguous, aligned segment(s), the hardware satisfies the whole warp with one (or a few) transactions.

Uncoalesced access: if the addresses are scattered, the hardware must issue a separate transaction per unique segment touched – potentially up to 32 transactions for 32 threads.

You're not saving bandwidth by reading less data – you're saving transactions, and each transaction has fixed overhead regardless of how much of it you actually use. The classic good case: stride-1 access

cpp

```
__global__ void copy(float *in, float *out) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    out[i] = in[i];
}
```

Thread 0 accesses in[0], thread 1 accesses in[1], ..., thread 31 accesses in[31]. Since float is 4 bytes, that's 128 contiguous bytes – exactly matching a 128-byte transaction, aligned if the array starts at a proper boundary. This is perfectly coalesced: one transaction serves the whole warp.

The classic bad case: strided access

cpp

```
__global__ void stridedCopy(float *in, float *out, int stride) {
    int i = (blockIdx.x * blockDim.x + threadIdx.x) * stride;
    out[i] = in[i];
}
```

If stride = 32, consecutive threads now access addresses 128 bytes apart. Each thread lands in a different memory segment, so the hardware may need up to 32 separate transactions to service one warp – you're using only a tiny fraction of each fetched chunk. Bandwidth utilization craters.

Where this bites people: 2D arrays / matrices

This is the most common real-world trap. Say you have a matrix stored row-major and you access it as `matrix[row][col]`:

cpp

```
// Good: consecutive threads vary `col` → consecutive memory addresses
int col = blockIdx.x * blockDim.x + threadIdx.x;
float val = matrix[row * width + col];    // coalesced
```

```
// Bad: consecutive threads vary `row` → addresses jump by `width` each time
int row = blockIdx.x * blockDim.x + threadIdx.x;
float val = matrix[row * width + col];    // strided, uncoalesced
```

The fix is almost always about how you map `threadIdx.x` to your data layout – make sure consecutive `threadIdx.x` values correspond to consecutive memory addresses, not consecutive rows.

This is exactly why matrix transpose kernels are a classic teaching example: reading row-major and writing column-major (or vice versa) means one of the two directions is inherently uncoalesced, which is why tiled shared-memory transpose kernels exist – you do one coalesced read, transpose in fast shared memory, then do one coalesced write.

Alignment matters too

Even stride-1 access can be uncoalesced if the starting address isn't aligned to the transaction boundary. If your array starts at an offset (e.g., you're accessing `array[1]` through `array[32]` instead of `array[0]` through `array[31]`), the warp's requests can straddle two segments instead of fitting cleanly in one, costing an extra transaction. `cudaMalloc` returns aligned memory, but sub-offsets and struct padding can break this.

Structs: Array-of-Structs vs. Struct-of-Arrays

cpp

```
// AoS - bad for coalescing
struct Particle { float x, y, z; };
Particle particles[N];
// thread i accessing particles[i].x skips over y, z of every particle – strided
```

```
// SoA - good for coalescing
struct Particles { float x[N], y[N], z[N]; };
// thread i accessing x[i] is stride-1 across the warp
```

This is a very common real-world refactor when optimizing CUDA code – SoA layouts are usually far more coalescing-friendly than AoS, even though AoS often feels more natural in CPU code.

How to check it

Nsight Compute (ncu) will directly report metrics like `gld_efficiency` / global load efficiency, or in newer versions, sector-to-request ratios – this tells you how much of each fetched transaction was actually useful.

A quick heuristic: if your kernel is memory-bound and slower than expected, check whether `threadIdx.x` maps to contiguous addresses in your access pattern first, before looking at anything else.

Want to go into shared memory next (it interacts closely with coalescing – e.g., the

tiled transpose pattern), or into bank conflicts, which are shared memory's version of this same problem?